

Advanced JavaScript Project Assignments

Project 1: GitHub Developer Finder

Story:

Your company is building a platform where recruiters can quickly search software developers and review their GitHub profiles before scheduling interviews.

API:

<https://api.github.com/users/{username}>

Example:

<https://api.github.com/users/octocat>

Objective:

Create a GitHub profile search application.

User Flow:

1. User enters a GitHub username.
2. Clicks Search.
3. Application fetches profile data from GitHub.
4. Profile details are displayed beautifully.

Display:

- Profile Photo
- Name
- Username
- Bio
- Followers
- Following
- Public Repositories
- Profile Link

Additional API:

<https://api.github.com/users/{username}/repos>

Bonus Features:

- Display latest 5 repositories.
- Search history using Local Storage.
- Loading Spinner.

- Error handling for invalid usernames.

Concepts Used:

Fetch API, Async/Await, Promises, Error Handling, DOM, Local Storage.

Expected Outcome:

Students should learn how real applications communicate with external APIs and handle success/failure scenarios.

Project 2: Smart Weather Dashboard

Story:

A travel company wants a weather dashboard that allows users to check weather conditions before planning a trip.

API:

OpenWeather API

<https://openweathermap.org/api>

Objective:

Build a weather forecasting application.

Input:

City Name

Display:

- City
- Country
- Temperature
- Feels Like Temperature
- Humidity
- Wind Speed
- Weather Condition
- Weather Icon

Requirements:

1. Search weather by city.
2. Show loading state while data is being fetched.
3. Handle invalid city names.
4. Store last 5 searched cities in Local Storage.

Bonus Features:

- Dark Mode.
- Display current date and time.
- Weather cards for multiple cities.

Concepts Used:

Async/Await, Fetch API, Promises, Local Storage, DOM Manipulation.

Expected Outcome:

Students learn API integration and asynchronous programming in a practical scenario.

Project 3: University Student Management Portal

Story:

Sharda University wants an internal portal to manage student records digitally.

Objective:

Create a complete Student Management System.

Student Data:

- Student ID
- Name
- Email
- Course
- Mobile Number

Requirements:

1. Add Student.
2. Edit Student.
3. Delete Student.
4. Search Student by Name.
5. Filter by Course.
6. Display Total Students.
7. Display Course-wise Statistics.

Technical Requirements:

Create a Student Constructor Function.

Methods using Prototype:

- getDetails()
- updateCourse()

- updateEmail()

Store all records using Local Storage.

Dashboard:

- Total Students
- MERN Students
- Java Students
- Python Students

Bonus Features:

- Export student data.
- Sort students alphabetically.
- Attendance percentage field.

Concepts Used:

Constructor Functions, Prototype, Objects, Arrays, Local Storage, DOM.

Expected Outcome:

Students understand how JavaScript OOP concepts are used in real applications.

Project 4: Personal Expense Tracker

Story:

Most people spend money daily but fail to track where their money goes. Build an Expense Tracker to help users manage finances.

Expense Fields:

- Title
- Amount
- Category
- Date

Categories:

Food
Travel
Shopping
Education
Bills
Other

Requirements:

1. Add Expense.
2. Edit Expense.
3. Delete Expense.
4. Search Expense.
5. Filter by Category.
6. Display Total Expenses.
7. Display Total Amount Spent.
8. Display Highest Expense.
9. Display Lowest Expense.

Local Storage:

All expenses must remain available after page refresh.

Dashboard:

- Total Records
- Monthly Expense
- Category-wise Expense

Bonus Features:

- Pie Chart
- Monthly Report
- Download Report

Concepts Used:

DOM, Arrays, Higher Order Functions, Local Storage, Async Simulation using setTimeout.

Expected Outcome:

Students learn CRUD operations and dashboard building.

Project 5: Job Application Tracker

Story:

Students apply to dozens of companies but often forget application status. Build a Job Application Tracker.

Application Fields:

- Company Name
- Job Role
- Location
- Application Date

- Status

Status Options:

- Applied
- Interview Scheduled
- Selected
- Rejected

Requirements:

1. Add Application.
2. Edit Application.
3. Delete Application.
4. Search Company.
5. Filter by Status.
6. Sort by Date.
7. Display Dashboard Statistics.

Dashboard:

- Total Applications
- Selected Applications
- Rejected Applications
- Pending Applications

Advanced Requirement:

Simulate server communication using Promises and setTimeout.

Example:

When adding application:

- Show Loading...
- Save after 2 seconds delay

Bonus Features:

- Success Rate Percentage
- Company-wise Analytics
- Recent Activity Section

Concepts Used:

Promises, Async/Await, Callbacks, setTimeout, Local Storage, DOM.

Expected Outcome:

Students gain practical understanding of asynchronous JavaScript and application state management.